

What is Component hierarchy,
State management strategy,
Rounding algorithm strategy,
Validation matrix,
Local storage schema

Explain about all

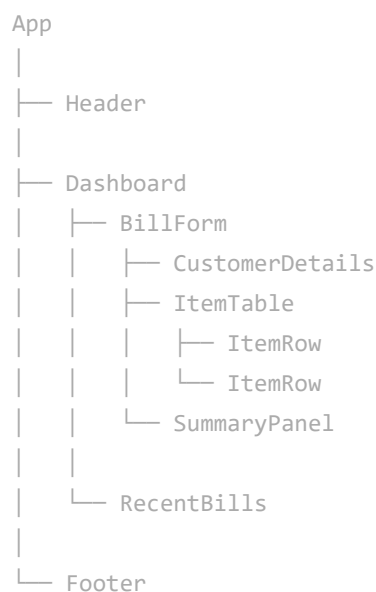
These are software architecture and design concepts. In a promptathon or real-world project, defining them before coding often makes the difference between a prototype and a production-ready solution.

1. Component Hierarchy

Component hierarchy is the **structure of your frontend UI components and how they are nested inside each other**.

Think of it as the family tree of your website.

Example: Smart Bill App



Why it matters

Without hierarchy:

- Components become huge
- Hard to maintain
- Logic gets duplicated

With hierarchy:

- Reusable components
- Easier debugging
- Easier scaling

Example

Instead of:

JavaScript

BillPage

containing 1000 lines,

split into:

JavaScript

BillPage

├─ ItemTable

├─ SummaryPanel

├─ CustomerSection

└─ SaveButton

Each component has a single responsibility.

2. State Management Strategy

State = data that changes while the app is running.

Examples:

- Customer name
- Quantity
- Tax %
- Theme
- Login status



Bad strategy

Every component stores its own copy.

```
ItemRow1 total
ItemRow2 total
Summary total
```

Now keeping them synchronized becomes painful.

Good strategy

Keep a **single source of truth**.

Example:

JavaScript

```
const bill = {
  customerName: "",
  items: []
}
```

Everything derives from this.

Typical strategy

Local Component State

Only used by one component.

JavaScript

```
isModalOpen
```

Shared State

Used by multiple components.



JavaScript

```
currentBill
```

Store higher up.

Global State

Used everywhere.

JavaScript

theme

user

Store centrally.

Smart Bill Example

BillForm State

```
|
|— Customer Data
|— Items
|— Discount
|— Tax
```

SummaryPanel reads from it.

ItemTable updates it.

Everything stays synchronized.

3. Rounding Algorithm Strategy

This is extremely important for financial software.

Imagine:

99.999999

appears due to floating point calculations.

Customer sees:

₹99.999999

Looks broken.



Define rules beforehand

Example:

Currency

Always:

2 decimal places

JavaScript

99.999

→ 100.00

Quantity

Maybe:

3 decimals

JavaScript

1.23456

→ 1.235

Tax

Example:

$\text{GST} = \text{subtotal} \times 18\%$

Decide:

1. Round every line item?
2. Round only final tax?

Different choices produce different totals.



Real-world strategy

Store full precision internally

Display rounded values only

Example:

Actual:
99.999999

Display:
100.00

This prevents cumulative errors.

4. Validation Matrix

A validation matrix is a complete list of:

What field exists and what rules apply to it.

Example

Field	Required	Min	Max	Format
Customer Name	Yes	2 chars	100 chars	Text
Phone	No	10 digits	10 digits	Numeric
Quantity	Yes	1	10000	Number
Price	Yes	0	1000000	Currency

Why it matters

Without validation:



Quantity = -5

accepted.

Or:

Price = abc

accepted.

Then calculations break.

Smart Bill Validation Matrix

Input	Rule
Customer Name	Required
Item Name	Required
Quantity	>0
Price	≥0
GST	0-100
Discount	≥0

Judge Perspective

Promptathon judges love seeing validation matrices because it shows:

- Edge-case thinking
- Production readiness
- Data quality awareness

5. Local Storage Schema

Local storage schema defines:

What data is stored in the browser and in what structure.



Bad

JavaScript

```
localStorage.setItem(  
  "data",  
  "random string"  
)
```

Nobody knows what's inside.

Good

JSON

```
{  
  "draftBill": {  
    "customerName": "Ashutosh",  
    "items": [  
      {  
        "name": "Pen",  
        "qty": 2,  
        "price": 10  
      }  
    ],  
    "tax": 18  
  }  
}
```

Structured and predictable.

Example Schema

JSON

```
{  
  "appVersion": "1.0",  
  "theme": "light",  
  
  "draftBill": {  
    "customerName": "",  
    "items": [],  
    "discount": 0,  
    "tax": 18  
  },  
  
  "settings": {  
    "currency": "INR",  

```




```
    "roundingMode": "HALF_UP"  
  }  
}
```

Why These 5 Matter Together

A strong architecture document often looks like:

- 1. Component Hierarchy**
 - How UI is organized
- 2. State Management Strategy**
 - Where data lives
- 3. Validation Matrix**
 - What inputs are allowed
- 4. Rounding Strategy**
 - How calculations stay accurate
- 5. Local Storage Schema**
 - How data persists

These are exactly the kinds of details that make judges think:

"This isn't just a generated website. Someone actually designed a deployable product."

